

Experiment-1 — Analysis of Linear Separability

AIM – To analyse the linear separability of the Iris dataset using visualisation and logistic regression.

Dataset – Iris Dataset

- Rows : 150
- Features : 4
- Classes : 3
- Target : target
- Type : multiclass
- Dataset is balanced
- Missing value : none

Features –

- Sepal length
- Sepal width
- Petal length
- Petal width

Code –

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

iris = load_iris()
df = pd.DataFrame(iris.data,
    columns=['Sepal-length', 'Sepal-width', 'Petal-length', 'Petal-width'])
df['Species'] = ['Iris-setosa' if x == 0
    else 'Iris-versicolor' if x == 1
    else 'Iris-virginica'
    for x in iris.target]

sns.scatterplot(
    data=df,
    x='Petal-length',
    y='Petal-width',
    hue='Species'
)
plt.xlabel('Petal length')
plt.ylabel('Petal width')
plt.title('Linear separability of Iris dataset')
plt.show()

X = df[['Petal-length', 'Petal-width']]
y = df['Species']
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y)
model = LogisticRegression(max_iter=200)
model.fit(X_train, y_train)
```

```
y_pred = model.predict(X_test)
```

```
print('Accuracy:', accuracy_score(y_test, y_pred))
```

Result – The Iris dataset was successfully analysed for linear separability using scatter plot & logistic regression.

Output –

Accuracy: 0.9666666666666667

A scatter plot is displayed in which Iris-setosa forms a separate cluster.

Experiment-2 — Find-S Algorithm

AIM – To implement the Find-S algorithm to find the most specific hypothesis from the Iris classification dataset.

Dataset – Iris dataset

- Rows : 150
- Feature : 4
- Classes : 3
- Target : target
- Type : multiclass
- For Find-S we use setosa = Yes and other = No

Feature

- Sepal length
- Sepal width
- Petal length
- Petal width

Theory – Find-S starts with the most specific hypothesis and generalises it when a positive example does not match the current hypothesis.

Code –

```
import pandas as pd
```

```
from sklearn.datasets import load_iris
```

```
iris = load_iris()
```

```
df = pd.DataFrame(iris.data,  
    columns=['Sepal-length', 'Sepal-width', 'Petal-length', 'Petal-width'])
```

```
df['Species'] = ['Iris-setosa' if x == 0  
    else 'Iris-versicolor' if x == 1  
    else 'Iris-virginica'  
    for x in iris.target]
```

```
features = ['Sepal-length', 'Sepal-width', 'Petal-length', 'Petal-width']
```

```
data = df.copy()
```

```
for feature in features:
```

```
    median_value = data[feature].median()  
    data[feature] = data[feature].apply(  
        lambda x: 'High' if x >= median_value else 'Low'  
    )
```

```
positive_data = data[data['Species'] == 'Iris-setosa']
```

```
hypothesis = list(positive_data.iloc[0][features])
```

```
for i in range(1, len(positive_data)):
```

```
    example = list(positive_data.iloc[i][features])
```

```

for j in range(len(hypothesis)):
    if hypothesis[j] != example[j]:
        hypothesis[j] = '?'

```

```

print('Final hypothesis:')
print(hypothesis)

```

Result – The Find-S algorithm was successfully implemented on the Iris dataset and the most specific hypothesis was obtained.

Output –

Final hypothesis:

```
['?', '?', 'Low', 'Low']
```

Experiment-3 — Candidate Elimination Algorithm

AIM – To implement the candidate elimination algorithm to find the possible hypotheses from the Iris classification dataset.

Dataset – Iris dataset

- Rows : 150
- Features : 4
- Target : Class
- Classification : Binary
- Setosa = Yes
- Other = No

Feature used –

- Sepal length
- Sepal width
- Petal length
- Petal width

Theory – Candidate elimination maintains two boundaries:

- S – specific boundary
- G – general boundary

It retains the hypotheses that are consistent with the training examples.

Code

```
import pandas as pd
```

```
from sklearn.datasets import load_iris
```

```
iris = load_iris()
```

```
df = pd.DataFrame(iris.data,
```

```
    columns=['Sepal-length', 'Sepal-width', 'Petal-length', 'Petal-width'])
```

```
df['Species'] = ['Iris-setosa' if x == 0
```

```
    else 'Iris-versicolor' if x == 1
```

```
    else 'Iris-virginica'
```

```
    for x in iris.target]
```

```
features = ['Sepal-length', 'Sepal-width', 'Petal-length', 'Petal-width']
```

```
data = df.copy()
```

```
for feature in features:
```

```
    data[feature] = pd.qcut(data[feature], q=3,
```

```
        labels=['Low', 'Medium', 'High']).astype(str)
```

```
data['Class'] = ['Yes' if s == 'Iris-setosa' else 'No' for s in data['Species']]
```

```

S = ['0'] * len(features)
G = [['?'] * len(features)]

for _, row in data.iterrows():
    x = list(row[features])
    label = row['Class']

    if label == 'Yes':
        for i in range(len(S)):
            if S[i] == '0':
                S[i] = x[i]
            elif S[i] != x[i]:
                S[i] = '?'
        G = [g for g in G
              if all(g[i] == '?' or g[i] == x[i] for i in range(len(features)))]

    else:
        new_G = []
        for g in G:
            if all(g[i] == '?' or g[i] == x[i] for i in range(len(features))):
                for i in range(len(features)):
                    if g[i] == '?' and S[i] not in ('?', '0') and S[i] != x[i]:
                        h = g.copy()
                        h[i] = S[i]
                        new_G.append(h)
            else:
                new_G.append(g)
        G = new_G

```

```

print('Specific Boundary (S):')
print(S)
print("\nGeneral Boundary (G):")
print(G)

```

Result – The candidate elimination algorithm was implemented, and the specific and general hypothesis boundaries were obtained.

Output –

Specific Boundary (S):

['?', '?', 'Low', 'Low']

General Boundary (G):

[['?', '?', 'Low', '?'], ['?', '?', '?', 'Low']]

Experiment-4 — Naive Bayes

AIM – To implement the Naive Bayes classification algorithm on the Iris dataset and evaluate its performance.

Dataset – Iris Dataset

- Rows : 150
- Features : 4
- Classes : 3
- Target : Species
- Type : Multiclass

- Missing values : None

Features

1. Sepal Length
2. Sepal Width
3. Petal Length
4. Petal Width

Theory – Naive Bayes is a supervised classification algorithm based on Bayes' theorem. It assumes that the features are conditionally independent given the class.

Bayes' theorem:

$$P(C|X) = P(X|C) P(C) / P(X)$$

For continuous numerical features, Gaussian Naive Bayes can be used.

Code –

```
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
```

```
iris = load_iris()
df = pd.DataFrame(iris.data,
                  columns=['SepalLength', 'SepalWidth', 'PetalLength', 'PetalWidth'])
df['Species'] = ['Iris-setosa' if x == 0
                 else 'Iris-versicolor' if x == 1
                 else 'Iris-virginica'
                 for x in iris.target]
```

```
X = df.drop('Species', axis=1)
y = df['Species']
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y)
```

```
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

```
model = GaussianNB()
model.fit(X_train_scaled, y_train)
y_pred = model.predict(X_test_scaled)
```

```
accuracy = accuracy_score(y_test, y_pred)
print('Accuracy:', accuracy)
print('Accuracy percentage:', accuracy * 100, '%')
print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))
print("\nClassification Report:")
print(classification_report(y_test, y_pred))
```

Result – The Naive Bayes classifier was successfully implemented on the Iris dataset with an accuracy of 96.67%.

Output –

Accuracy: 0.9666666666666667

Accuracy percentage: 96.66666666666667 %

Confusion Matrix:

```
[[10 0 0]
 [ 0 9 1]
 [ 0 0 10]]
```

Classification Report:

	precision	recall	f1-score	support
Iris-setosa	1.00	1.00	1.00	10
Iris-versicolor	1.00	0.90	0.95	10
Iris-virginica	0.91	1.00	0.95	10
accuracy		0.97		30
macro avg	0.97	0.97	0.97	30
weighted avg	0.97	0.97	0.97	30

Experiment-5 — ID3 Decision Tree

AIM – To implement the ID3 Decision Tree classification algorithm on the Iris dataset and evaluate its classification performance.

Dataset – Iris dataset

- Rows : 150
- Feature : 4
- Classes : 3
- Target : Species
- Type : multiclass
- Missing value : None

Features

1. Sepal Length
2. Sepal Width
3. Petal Length
4. Petal Width

Theory – ID3 is a supervised learning algorithm used for classification. It constructs a decision tree by selecting the feature that provides the highest information gain.

Entropy is calculated as:

$$\text{Entropy}(S) = - \sum p_i \log_2(p_i)$$

Information Gain is:

$$\text{Gain}(S, A) = \text{Entropy}(S) - \sum_v (|S_v| / |S|) \text{Entropy}(S_v)$$

The attribute having the highest information gain is selected for splitting.

Code –

```
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, export_text
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

iris = load_iris()
df = pd.DataFrame(
    iris.data,
    columns=['SepalLength', 'SepalWidth', 'PetalLength', 'PetalWidth'])
df['Species'] = ['Iris-setosa' if x == 0
```

```

else 'Iris-versicolor' if x == 1
else 'Iris-virginica'
for x in iris.target]

```

```

X = df.drop('Species', axis=1)
y = df['Species']
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y)

```

```

model = DecisionTreeClassifier(
    criterion='entropy',
    random_state=42)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

```

```

accuracy = accuracy_score(y_test, y_pred)
print('Accuracy:', accuracy)
print('Accuracy percentage:', accuracy * 100, '%')
print(confusion_matrix(y_test, y_pred))
print(classification_report(y_test, y_pred))
print(export_text(model, feature_names=list(X.columns)))

```

Result – The ID3 decision tree algorithm was successfully implemented using entropy as the splitting criterion, and the model achieved an accuracy of 93.33% on the dataset.

Output –

```

Accuracy: 0.9333333333333333
Accuracy percentage: 93.33333333333333 %
[[10 0 0]
 [ 0 9 1]
 [ 0 1 9]]

```

Experiment-6 — K-Nearest Neighbors

AIM – To implement the KNN supervised learning algorithm on the Iris classification dataset & evaluate its performance.

Dataset – Iris Dataset

- Rows : 150
- Features : 4
- Classes : 3
- Target : Species
- Type : Multiclass
- Missing Values : None

Features –

1. Sepal Length
2. Sepal Width
3. Petal Length
4. Petal Width

Theory – For classification, KNN finds the K nearest training samples to a new data point and assigns the class that occurs most frequently among those neighbours.

The Euclidean distance between two points is:

$$d = \sqrt{\sum_{i=1}^n (x_i - y_i)^2}$$

Let K = 5.

Before applying KNN, the features are standardized because the algorithm depends on distance calculation.

Code –

```
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
```

```
iris = load_iris()
df = pd.DataFrame(iris.data,
    columns=['SepalLength', 'SepalWidth', 'PetalLength', 'PetalWidth'])
df['Species'] = ['Iris-setosa' if x == 0
    else 'Iris-versicolor' if x == 1
    else 'Iris-virginica'
    for x in iris.target]
```

```
X = df.drop('Species', axis=1)
y = df['Species']
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y)
```

```
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

```
model = KNeighborsClassifier(n_neighbors=5)
model.fit(X_train_scaled, y_train)
y_pred = model.predict(X_test_scaled)
```

```
accuracy = accuracy_score(y_test, y_pred)
print('Accuracy:', accuracy)
print('Accuracy percentage:', accuracy * 100, '%')
print(confusion_matrix(y_test, y_pred))
print(classification_report(y_test, y_pred))
```

Result – The K-Nearest Neighbors algorithm was successfully implemented on the Iris dataset, and the classifier achieved an accuracy of 93.33% on the dataset.

Output –

Accuracy: 0.9333333333333333

Accuracy percentage: 93.33333333333333 %

```
[[10 0 0]
```

```
 [ 0 10 0]
```

```
 [ 0 2 8]]
```

```
precision recall f1-score support
```

Iris-setosa	1.00	1.00	1.00	10
Iris-versicolor	0.83	1.00	0.91	10
Iris-virginica	1.00	0.80	0.89	10

accuracy		0.93	30
----------	--	------	----

macro avg	0.94	0.93	0.93	30
-----------	------	------	------	----

weighted avg	0.94	0.93	0.93	30
---------------------	-------------	-------------	-------------	-----------